UNIVERSITY OF DHAKA

RedGrid: Dependency-Constrained UCB Exploration for Autonomous Penetration Testing

Submitted by

Class Roll: 50028
Registration No: H-55

Class Roll: 50040
Registration No: H-49

Submitted to the
Department of Computer Science and Engineering
University of Dhaka, Bangladesh
in partial fulfillment of the requirements for the degree of
Professional Masters in Information and Cyber Security (PMICS).

Declaration Form

University of Dhaka

The work presented in this inception report, titled "**RedGrid: Dependency-Constrained UCB Exploration for Autonomous Penetration Testing**", has been carried out independently by the undersigned candidates under the academic supervision of Dr. Md. Abdur Razzaque. Each of the authors affirms that, except where explicitly attributed to the published work of others, all content — including the literature review, gap analysis, proposed architecture, and evaluation plan — reflects their own intellectual effort. Every source drawn upon has been identified and cited in full accordance with standard academic referencing practice. No portion of this report has been submitted, either in whole or in part, towards any other academic qualification at this institution or elsewhere.

Name of the Candidate
Nishan Paul

Name of the Candidate
Md Rakibur Rahman

Dr. Md. Abdur Razzaque
Professor and Chair
Department of Computer Science and Engineering
University of Dhaka

"The only truly secure system is one that is powered off, cast in a block of concrete and sealed in a lead-lined room with armed guards — and even then I have my doubts."

Eugene H. "Gene" Spafford
Professor of Computer Science, Purdue University

UNIVERSITY OF DHAKA

Abstract

LLM-orchestrated autonomous penetration testing systems face two quantitatively documented failure modes: insufficient exploration (CVE-Bench records 37.5%–80.0% exploration-failure rates; the best agent achieves only 13% pass@5 on 40 critical-severity CVEs) and weak prerequisite reasoning between discovered weaknesses (PentestEval's Attack Decision-Making stage averages a Spearman correlation of 0.25 across nine LLMs; expert-quality ADM alone raises end-to-end success from 0.31 to 0.67). No reviewed system combines open-ended attack-surface discovery with a dynamically constructed dependency model, and none has been evaluated across more than one attack-surface family. This report documents the first stage of RedGrid — a four-layer orchestration hierarchy, Dual-Layer World Model, UCB-guided Vulnerability Dependency Graph, and Cross-Mission Memory — to be evaluated against CVE-Bench, PentestEval, PrediQL, and MHBench; no results are available at this stage.

Acknowledgements

Sincere gratitude is owed first to Dr. Md. Abdur Razzaque, Professor and Chair of Department of Computer Science and Engineering, University of Dhaka, whose willingness to take on this project at its earliest and most uncertain stage made everything that follows possible. His insistence that a credible research question must be grounded in a specific, measurable gap — and not merely in a general area of interest — redirected the initial scope of this work at a critical moment and gave the thesis its present focus. Beyond that early intervention, the patience and intellectual generosity he has shown in guiding two students through the literature are debts that will take the full thesis to begin to repay. Acknowledgement is also owed to the researchers whose published work forms the foundation on which this thesis builds: the authors of CVE-Bench, PentestEval, HPTSA, PentestGPT, VulnBot, CHECKMATE, Incalmo, and PrediQL, each of whom designed evaluations rigorous enough to yield the kind of specific, falsifiable failure-mode evidence that makes a thesis like this possible to motivate honestly.

Contents

Declaration Form i

Abstract iii

Acknowledgements iv

List of Figures vii

List of Tables viii

1 Introduction 1
- 1.1 Background and Motivation . . . 1
  - 1.1.1 Cybersecurity and Vulnerability Assessment . . . 1
  - 1.1.2 Why Conventional VAPT Has Limits . . . 2
  - 1.1.3 Large Language Models and AI Agents . . . 2
  - 1.1.4 The Emergence of Autonomous VAPT Research . . . 3
  - 1.1.5 Motivation for This Thesis . . . 4
- 1.2 Problem Statement . . . 5
- 1.3 Objectives . . . 8
- 1.4 Scope of the Project . . . 10
- 1.5 Expected Contributions . . . 11
- 1.6 Challenges . . . 13
- 1.7 Organization . . . 16

2 Literature Review and Related Work 18
- 2.1 Domain Overview . . . 18
- 2.2 Existing Systems . . . 19
  - 2.2.1 Early Single-Agent Approaches . . . 21
  - 2.2.2 Multi-Agent and Team-Based Orchestration . . . 22
  - 2.2.3 Planning-Augmented Approaches . . . 23
  - 2.2.4 Non-Web Attack Surfaces and Benchmarks . . . 23
- 2.3 Comparative Analysis . . . 24
- 2.4 Research Gap . . . 27
- 2.5 Summary . . . 29

3 Proposed Methodologies 30

3.1 System Overview 30
3.2 System Requirements 30
3.3 Proposed Architecture 30
3.4 System Workflow 30
3.5 Tools and Technologies 30

4 Execution Plan **31**
4.1 Work Breakdown Structure 31
4.2 Project Timeline 31
4.3 Milestones 31
4.4 Resource Requirements 31
4.5 Risk Assessment 31

5 Experimental Results **32**
5.1 Evaluation Plan 32

6 Conclusions **33**
6.1 Summary 33
6.2 Expected Deliverables 33
6.3 Future Works 33

Bibliography **34**

List of Figures

1.1 Exploration-failure rate as the dominant CVE-Bench failure mode, by agent and evaluation setting. T-Agent is the hierarchical multi-agent framework from Zhu et al. [1]; data from Table 5 of Zhu et al. [2]. 6

List of Tables

1.1 PentestEval ground-truth-injection ablation: cumulative end-to-end success rate as expert ground truth replaces each stage's LLM output, from left to right [3]. 7
1.2 Attack surfaces in scope, their primary governing benchmarks, and representative vulnerability types. 10

2.1 Eleven papers selected for review, with their main focus and key relevance to this research. 20
2.2 Comparison of systems and benchmarks reviewed, across four recurring architectural dimensions. 25
2.3 Three-dimensional gap in the reviewed autonomous VAPT literature: what each capability area shows and where it falls short. 27

Acronym	**W**hat (it) **S**tands **F**or
ADM	**A**ttack **D**ecision-**M**aking (PentestEval stage 4)
CSRF	**C**ross-**S**ite **R**equest **F**orgery
CVE	**C**ommon **V**ulnerabilities and **E**xposures
CVSS	**C**ommon **V**ulnerability **S**coring **S**ystem
EG	**E**xploit **G**eneration (PentestEval stage 5)
ER	**E**xploit **R**evision (PentestEval stage 6)
FSM	**F**inite-**S**tate **M**achine
GPT	**G**enerative **P**re-trained **T**ransformer
GT	**G**round **T**ruth
HPTSA	**H**ierarchical **P**lanning and **T**ask-**S**pecific **A**gents
HTB	**H**ack**T**he**B**ox
IC	**I**nformation **C**ollection (PentestEval stage 1)
IDOR	**I**nsecure **D**irect **O**bject **R**eference
LFI	**L**ocal **F**ile **I**nclusion
LLM	**L**arge **L**anguage **M**odel
PDDL	**P**lanning **D**omain **D**efinition **L**anguage
PTG	**P**enetration **T**esting **G**raph
PTT	**P**enetration **T**esting **T**ree
RCE	**R**emote **C**ode **E**xecution
ReAct	**Re**ason + **Act** (agent interaction framework)
REST	**Re**presentational **S**tate **T**ransfer
SMP	**S**equential **M**odular **P**ipeline (PentestEval baseline)
SQL	**S**tructured **Q**uery **L**anguage
SQLi	**SQL** **I**njection
SSRF	**S**erver-**S**ide **R**equest **F**orgery
SSTI	**S**erver-**S**ide **T**emplate **I**njection
UCB	**U**pper **C**onfidence **B**ound
VAPT	**V**ulnerability **A**ssessment and **P**enetration **T**esting
VDG	**V**ulnerability **D**ependency **G**raph
WF	**W**eakness **F**iltering (PentestEval stage 3)
WG	**W**eakness **G**athering (PentestEval stage 2)
XSS	**C**ross-**S**ite **S**cripting

To our parents, who taught us before any classroom could.

Chapter 1

Introduction

1.1 Background and Motivation

1.1.1 Cybersecurity and Vulnerability Assessment

Modern organisations depend on networked software systems for nearly every operational function, and the security of those systems has become a central concern. A security vulnerability is a flaw in software, configuration, or design that an attacker can exploit to gain unauthorised access, execute arbitrary code, disrupt service, or exfiltrate data. Vulnerabilities are catalogued in the Common Vulnerabilities and Exposures (CVE) database after disclosure; the interval between disclosure and remediation deployment frequently extends to months in production environments, leaving systems exposed to any attacker who can reconstruct an exploit from the published description.

Vulnerability Assessment and Penetration Testing (VAPT) is the practice of proactively identifying and demonstrating these weaknesses before an adversary does. A penetration test is an authorised, systematic attempt to discover and exploit weaknesses in a target system, typically following a structured workflow: initial reconnaissance to understand the target's attack surface, identification of candidate weaknesses, exploitation of confirmed vulnerabilities, and reporting of findings with

remediation guidance. VAPT is widely recognised as an essential security practice, and demand for it has grown considerably as organisations increase their exposure through cloud services, APIs, and web applications.

1.1.2 Why Conventional VAPT Has Limits

Despite its importance, conventional penetration testing faces practical constraints. Skilled human testers are scarce and expensive: a thorough engagement against a moderately complex target demands days of expert effort and costs can reach thousands of dollars per assessment [4]. Scheduled, periodic testing leaves gaps during which newly introduced vulnerabilities remain undetected. Automated scanning tools — such as network port scanners and web application vulnerability scanners — can partially fill this gap, but they operate on fixed rule sets and signature databases and lack the contextual reasoning needed to chain together exploits. The practical limits of these tools are not merely theoretical: Fang et al. [5] tested the two most widely deployed open-source scanners (ZAP and Metasploit) against a benchmark of 15 real-world one-day vulnerabilities and found that both achieved 0% success, whereas a capable LLM agent succeeded on the same targets. Scanners cannot adapt to novel application logic, reason about prerequisite relationships between attack steps, or operate across the breadth of a modern attack surface without human guidance.

These limitations have driven a recent and rapid research interest in applying LLM-based agents to automate parts of the VAPT process — an interest that has produced a substantive body of work since 2023.

1.1.3 Large Language Models and AI Agents

Large Language Models (LLMs) are deep learning models trained on large corpora of text that have demonstrated a remarkable ability to follow instructions, reason through multi-step problems, write and execute code, and interact with external tools.

An **LLM agent** extends a base language model with the ability to take actions in an environment: calling tools (such as a web browser, a terminal, or an API), observing the results, and iterating toward a goal. The simplest agents follow a *Reason + Act* (ReAct) loop — reason about the current state, decide on an action, observe the outcome, and repeat. More sophisticated designs introduce hierarchical planning, role-specialised sub-agents, persistent memory, and structured validation.

A finding that recurs independently across six systems in the VAPT literature — and that directly shapes the design choices in this thesis — is that **architecture, not model scale, is the dominant variable in autonomous exploitation performance** [6]. Structured pipelines running comparatively inexpensive models consistently outperform unstructured loops running frontier models: Singer et al. [7] show that their multi-host framework with Claude Haiku 3.5 surpasses a strong single-agent baseline built on Claude Sonnet 4, and separate work demonstrates that GPT-4o-mini exceeds GPT-4o at web exploitation once a deterministic finite-state machine governs agent behaviour [6]. This architectural primacy motivates close attention to structural design rather than model selection in the work described here.

The emergence of capable LLM agents has opened a genuinely new question for cybersecurity: *can an LLM agent carry out penetration testing tasks autonomously, without step-by-step human direction?*

1.1.4 The Emergence of Autonomous VAPT Research

A growing body of research — developed between 2023 and 2025 — has answered parts of this question affirmatively, and in doing so has produced the quantitative baseline against which this thesis positions itself.

Early work by Fang et al. [8] showed that a single GPT-4 agent with browser access could autonomously exploit simplified web vulnerabilities, establishing that the basic capability exists. A later study by the same group collected a benchmark of 15 real-world one-day vulnerabilities and found that GPT-4, when given a CVE

description, achieved a pass@5 success rate of 87% — while every other tested model (GPT-3.5, eight open-source models) and every open-source scanner achieved 0% [5]. Crucially, without the CVE description, GPT-4's pass@5 rate collapsed to 7%, establishing an important early distinction: exploiting a vulnerability whose location and nature the agent already knows differs fundamentally from discovering an unknown vulnerability independently. This discovery gap is the central challenge that subsequent systems attempt to close.

Zhu et al. [1] addressed this gap directly by introducing a hierarchical team of planning and task-specific agents (HPTSA), which was the first system reported to autonomously exploit real zero-day vulnerabilities — vulnerabilities for which no CVE description is provided. HPTSA achieved a pass@5 rate of 42% and a pass@1 rate of 18% on a 14-CVE benchmark, representing a 4.3× improvement over a no-description single-agent baseline on pass@1. These figures are the primary comparative baseline for the evaluation described in this thesis.

Deng et al. [9] identified two recurring failure modes in single-agent systems: first, context loss over long sessions, which causes the agent to lose awareness of previously attempted attack paths (Finding 3); and second, a depth-first, recency-biased search pattern, in which the agent anchors to the most recently discussed surface and fails to return to earlier discoveries (Finding 4). These two modes together explain why single-agent systems consistently underperform on targets that require coordinated, multi-step attack chains. Later systems address these failure modes through multi-agent architectures [1, 4], declarative task abstractions [7], and planning-augmented designs [10].

1.1.5 Motivation for This Thesis

The starting point for this work was a close reading of this body of literature to determine whether it had already addressed the core challenges of autonomous VAPT, and if not, where exactly the remaining gaps lie. The review spans single-agent

exploitation studies, hierarchical multi-agent frameworks, planning-augmented approaches, and the principal benchmark suites used to assess autonomous VAPT agents (surveyed in full in Chapter 2).

Two recurring failure modes surfaced across independently developed systems: agents that search broadly over a target's attack surface do so without an explicit model of which vulnerabilities depend on which others; agents that reason about such dependencies do so on pre-enumerated, curated weakness sets that are not assembled dynamically during a live mission. These two failure modes appear to be complementary rather than coincidental, and they are the gaps this thesis has chosen to investigate. The investigation is at an early stage — the following chapters describe the direction, the proposed design, and the evidence that motivates it, rather than results already obtained.

1.2 Problem Statement

Autonomous VAPT systems face two recurring failure modes that have been independently measured across multiple benchmark studies. The two modes are complementary rather than coincidental: exploration breadth and dependency-aware planning are not separable capabilities. A system that explores widely without a model of which vulnerabilities depend on which others will revisit dead-end paths and waste its iteration budget on routes that are unreachable without prior steps. Conversely, a system that reasons about dependencies only over a pre-supplied weakness set cannot operate on an unfamiliar attack surface where no such set exists in advance. Both quantitative failure modes are described below.

Failure Mode 1 — Insufficient exploration. On CVE-Bench [2], a benchmark of 40 critical-severity (CVSS $\geq$ 9.0) real-world web-application CVEs, the strongest reported agent achieves a pass@5 success rate of only 13% (one-day setting) and 10% (zero-day setting). CVE-Bench's failure-mode breakdown (Table 5 of Zhu et al. 2) identifies *insufficient exploration* as the dominant cause of failure across every agent and evaluation setting, with exploration-failure rates ranging from 37.5% to 80.0%,

as illustrated in Figure 1.1. The failure is not merely a matter of insufficient effort: CVE-Bench defines it specifically as an agent's failure to locate the exploitable endpoint, even when given a high-level vulnerability description. Agents converge early on a narrow set of candidate paths and do not return to probe the remainder of the attack surface.

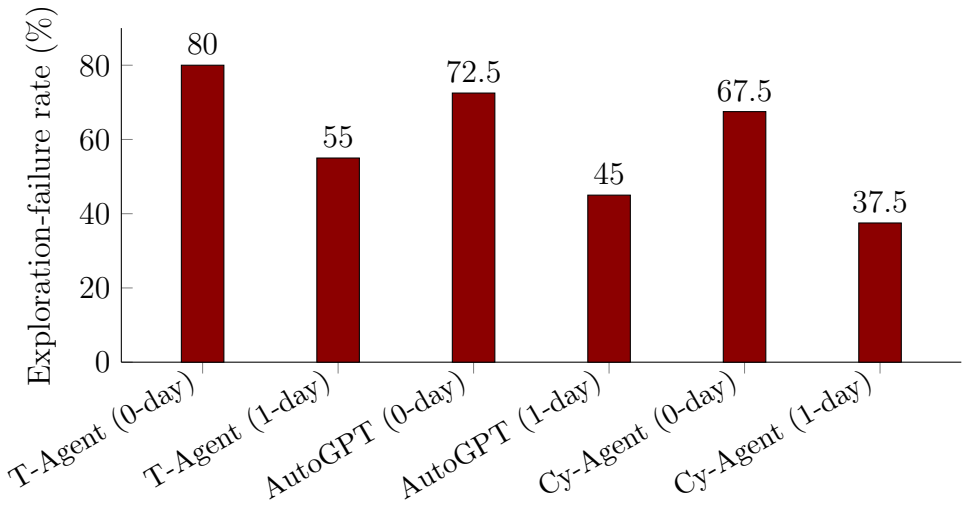

FIGURE 1.1: Exploration-failure rate as the dominant CVE-Bench failure mode, by agent and evaluation setting. T-Agent is the hierarchical multi-agent framework from Zhu et al. [1]; data from Table 5 of Zhu et al. [2].

Failure Mode 2 — The dependency-reasoning gap. PentestEval [3] decomposes the penetration testing pipeline into six sequential stages — Information Collection (IC), Weakness Gathering (WG), Weakness Filtering (WF), Attack Decision-Making (ADM), Exploit Generation (EG), and Exploit Revision (ER) — and evaluates nine state-of-the-art LLMs on each. The two weakest stages are WG (average Jaccard similarity 0.29 against expert ground truth) and ADM, the planning stage that requires reasoning about prerequisite relationships between candidate weaknesses. ADM's average Spearman rank correlation against expert judgement is 0.25, consistent across all nine evaluated models (range: 0.07 for GPT-3.5-Turbo to 0.34 for Qwen-Max): no tested model achieves meaningful alignment with expert attack-decision reasoning. The ground-truth-injection ablation in PentestEval quantifies how much end-to-end performance improves when each stage's output is replaced with expert annotations, applied cumulatively, as summarised in Table 1.1.

TABLE 1.1: PentestEval ground-truth-injection ablation: cumulative end-to-end success rate as expert ground truth replaces each stage's LLM output, from left to right [3].

Configuration	End-to-end success	Marginal gain
SMP (baseline, no ground truth)	0.31	–
+ GT Weakness Gathering (WG)	0.50	+0.19
+ GT Weakness Filtering (WF)	0.53	+0.03
+ GT Attack Decision-Making (ADM)	0.67	**+0.14**

Two observations from Table 1.1 are critical for motivating this thesis. First, the current end-to-end ceiling without any ground truth is 0.31 — the SMP baseline that simply executes the five stages in sequence with LLM outputs throughout. Second, ADM delivers the largest single-stage marginal gain (+0.14) of any stage, and this gain is measured on top of an already ground-truthed WG and WF pipeline, not in isolation. The gap between 0.31 and 0.67 — a 36-point range — is the portion of end-to-end performance attributable, in this benchmark, to better dependency reasoning alone. Any dynamically constructed dependency structure that an agent builds from its own discovery output, without expert annotation, will approximate rather than match the ground-truth ceiling, so the realistic target sits somewhere within this range rather than at its upper bound.

The compound gap. The two failure modes together identify a gap that appears unaddressed in the reviewed literature. Systems built for broad exploration of an unfamiliar attack surface — HPTSA [1] and VulnBot [4] — dispatch task lists from a central planner without an explicit model of which weaknesses must precede others. The one system in the reviewed literature that models dependencies explicitly — CHECKMATE [10], using PDDL-style planning operators — requires the operator to enumerate relevant weaknesses before the planning phase begins; it is not designed to operate on an unknown attack surface where no weakness list is available in advance. A third gap is cross-surface evaluability: every system in the reviewed set undergoes evaluation against a single attack-surface family, making it unclear how well any architectural decision generalises beyond the surface on which it was measured.

The problem this thesis investigates is therefore: *can a single LLM-orchestrated multi-agent architecture combine open-ended exploration of an unfamiliar attack surface with a dynamically constructed, prerequisite-aware dependency model, and if so, does the combination produce a measurable improvement in end-to-end task success when evaluated against independently published benchmarks spanning more than one attack-surface family?* Whether this is achievable within the constraints of a six-month research programme, and whether the improvement is large enough to be meaningful, are empirical questions that the remaining chapters describe a plan for answering.

1.3 Objectives

Guided by the three gaps identified in Section 1.2, the objectives below define the research questions this thesis investigates. They are working hypotheses, not completed results; the exact scope and design of each will be revised as implementation and early experiments provide evidence that the literature review alone cannot. Each objective maps to a specific measurable claim that will be evaluated against the benchmarks described in Section 1.4.

1. **Dependency-aware exploration via the VDG.** To investigate whether representing candidate vulnerabilities as a **Vulnerability Dependency Graph (VDG)** — a directed graph in which edges encode prerequisite relationships between candidate weaknesses, and node selection is governed by an Upper Confidence Bound (UCB) policy that balances exploitation of high-confidence paths with exploration of unvisited ones — produces a measurable improvement in pass@5 exploitation success over flat-dispatch baselines on CVE-Bench [2] and PentestEval [3].

2. **Dual-layer knowledge representation.** To design and implement a **Dual-Layer World Model** that cleanly separates what an agent has empirically confirmed about a target (the Evidence Layer, recording tool outputs and verified state facts) from what it has inferred but not yet confirmed (the Analysis

Layer, hosting the VDG and ordinal evidence scores). Separating these layers is intended to allow independent diagnosis of where a mission fails — during discovery, during dependency inference, or during exploit generation — and to prevent unverified hypotheses from contaminating the agent's operational state.

3. **Four-layer agent orchestration.** To specify and implement a four-layer orchestration hierarchy — Mission Planner (goal decomposition and strategy), Team Manager (phase coordination and sub-agent dispatch), Specialist Agents (surface-specific execution roles), and an Execution and Validation Layer (tool invocation and result parsing) — drawing on the architectural patterns that recur across the strongest-performing systems in the reviewed literature [1, 4, 7], and to evaluate whether this structure improves performance relative to both single-agent and flat multi-agent baselines.

4. **Cross-mission memory and transfer.** To examine whether a **Cross-Mission Memory** module — which retains and selectively replays successful attack strategies from prior engagements, keyed by target technology fingerprint — produces a measurable benefit on targets that share underlying technology, without introducing the negative-transfer risk that arises when a strategy valid against one version of a component is applied to an incompatible version. This objective is treated as secondary: whether it becomes a core research question or supporting infrastructure depends on what early design work reveals.

5. **Multi-surface evaluation.** To evaluate the full system against at least two independently published, oracle-backed benchmarks covering distinct attack-surface families — specifically CVE-Bench (web application CVEs) [2], PentestEval (stage-level penetration testing) [3], PrediQL (GraphQL APIs) [11], and Incalmo MHBench (multi-host Active Directory environments) [7] — with per-surface reporting that distinguishes shared architectural gains from surface-specific results, rather than reporting a single aggregated number.

1.4 Scope of the Project

Based on the literature reviewed in Chapter 2, this project bounds itself by one governing rule: work proceeds only with attack surfaces for which a dedicated, reusable, oracle-backed benchmark already exists in the reviewed literature, rather than constructing new benchmarks independently. This constraint keeps the evaluation grounded and reproducible, and it ensures that any performance claim this thesis makes can be independently verified against a fixed, published test suite. The scope may narrow further once early implementation provides more information about what is achievable within the remaining timeline. Table 1.2 summarises the three attack-surface families currently considered in scope, the primary benchmarks that govern each, and the vulnerability types each family contributes to the evaluation.

TABLE 1.2: Attack surfaces in scope, their primary governing benchmarks, and representative vulnerability types.

Attack surface	**Primary governing benchmark(s)**	**Representative vulnerability types**
Web application (HTTP/HTML)	CVE-Bench — 40 critical-severity CVEs, oracle-backed pass/fail [2]; PentestEval — 346 tasks across 12 real-world scenarios, stage-level scoring [3]	SQLi, XSS, CSRF, SSRF, SSTI, LFI, file-upload RCE, IDOR, authentication bypass, privilege escalation
GraphQL APIs	PrediQL — 6 real GraphQL APIs, dependency-aware oracle [11]	Schema abuse, dependency-chain injection, batched-auth bypass, IDOR
Multi-host / Active Directory	Incalmo MHBench — 40 networked environments, declarative task oracle [7]	Lateral movement, credential reuse, privilege escalation across host chains

Results from the single-agent and hierarchical-team baselines in Fang et al. [5] and Zhu et al. [1] — 87% and 42% pass@5 respectively on their own curated suites — serve as comparative reference points rather than primary evaluation targets: RedGrid is not re-evaluated on those closed suites, but their reported numbers contextualise what the state of the art was before the benchmarks listed in Table 1.2 existed.

What is currently left out. General REST API exploitation is not assessed directly: no standardised, reusable REST API benchmark comparable to CVE-Bench or PrediQL appears in the reviewed literature. Techniques from the REST API testing literature — dependency inference between API calls, response-feedback pruning — may inform internal design choices, but REST-API-specific performance claims are not part of the evaluation plan. Binary exploitation, physical- and network-layer attacks, and social engineering are also outside the current scope for the same reason: no oracle-backed benchmark for these surfaces appears in the reviewed literature, and the governing rule of this project requires one. Should a suitable benchmark emerge during the thesis period, the scope section will be revised accordingly.

1.5 Expected Contributions

The following reflects the direction this thesis considers worth pursuing, framed as working hypotheses rather than results already in hand. None of the items below has yet been implemented or empirically assessed; each represents a claim to be tested against the benchmarks described in Section 1.4. The framing will be revised as implementation and early experiments provide evidence the literature review alone cannot.

- **A proposed system architecture.** The primary architectural contribution is a four-layer orchestration hierarchy — Mission Planner, Team Manager, Specialist Agents, and Execution/Validation Layer — combined with a Dual-Layer World Model that separates empirically confirmed target state (Evidence Layer) from inferred but unconfirmed hypotheses (Analysis Layer, hosting the VDG). This design synthesises structural patterns that recur independently across the strongest-performing systems in the reviewed literature [1, 4, 7] and addresses the mismatch identified in Sections 1.2 and 1.3: no reviewed system combines both layers simultaneously. Whether this architecture delivers a performance improvement over flat-dispatch baselines is an

empirical question; the architecture itself, together with its formal specification, is a contribution independent of that outcome.

- **Dependency-aware exploration via the VDG.** The primary empirical hypothesis is that a **Vulnerability Dependency Graph (VDG)** — a dynamically constructed directed graph in which edges encode prerequisite relationships between candidate weaknesses, and node selection is governed by a UCB policy — improves end-to-end exploitation success relative to flat-dispatch baselines. The quantitative opportunity is bounded: the current best end-to-end baseline without ground truth is 0.31 (PentestEval SMP); the upper bound with perfect attack-decision-making is 0.67 (PentestEval GT-ADM ablation). The target is an improvement somewhere within this range, measured on CVE-Bench [2] and PentestEval [3]. No position on the expected magnitude of improvement is available with confidence at this stage.

- **Cross-mission memory and selective strategy reuse.** A secondary hypothesis is that a **Cross-Mission Memory** module — retaining and selectively replaying successful attack strategies from prior engagements, keyed by target technology fingerprint — produces a measurable benefit on targets sharing underlying technology, without inducing the negative-transfer risk discussed in Section 1.6. Whether this becomes a core research contribution or supporting infrastructure depends on what early design work reveals; it is not a prerequisite for the primary architectural and VDG contributions.

- **Multi-surface evaluation with separated per-surface reporting.** Independently of whether the above hypotheses hold, this thesis intends to evaluate the proposed system against at least two independently published benchmarks spanning distinct attack-surface families — CVE-Bench and PentestEval (web applications), PrediQL (GraphQL APIs) [11], and Incalmo MHBench (multi-host Active Directory environments) [7] — with standardised oracles and per-surface reporting. Every reviewed system undergoes evaluation on a single attack-surface family, making it impossible to distinguish architectural gains from surface-specific tuning effects. Multi-surface evaluation with separated

reporting is therefore a methodological contribution in itself, regardless of the numeric outcome.

The engineering infrastructure that enables the above — structured inter-agent communication protocols, retry-and-diagnose validation loops, deterministic tool-call logging, and benchmark harness integration — is a necessary precondition for reproducible evaluation rather than an independent research claim. It is listed here for completeness and will be documented in detail in Chapter 4.

1.6 Challenges

The challenges listed below surfaced repeatedly during the literature review and represent specific risks to the validity of the proposed research. They are stated here explicitly, rather than deferred to implementation, so that each can be assigned a mitigation strategy before work begins rather than after a problem is encountered.

1. **Inferring dependencies without ground truth.** The VDG's edges are constructed from LLM inference over live discovery output, not from expert annotation. This is the single riskiest component of the design: PentestEval [3] shows that even with a perfect, expert-supplied weakness set, the best LLM achieves only 0.34 Spearman correlation on attack decision-making. LLM-inferred edges on a noisy, dynamically discovered set will be less accurate still. *Mitigation:* before relying on VDG inference in any live evaluation, a pilot study will measure Spearman alignment between VDG-inferred dependency rankings and PentestEval's expert-annotated ground truth on Scenarios 1–12, and the resulting accuracy figure will be reported alongside all end-to-end results.

2. **Sandbox results may not transfer to the real world.** Fang et al. [8] found only 1 exploitable XSS in 50 real-world candidate sites (2%), against

73.3% in a matched sandbox. All evaluation in this thesis is against published, oracle-backed benchmarks rather than live production systems. *Mitigation:* every reported result will explicitly state the benchmark, its construction method, and whether it uses real deployed software (CVE-Bench, PrediQL) or purpose-built lab environments (PentestEval, MHBench), so that readers can calibrate the real-world transferability of any number.

3. **Hallucination in exploit generation and tool use.** CVE-Bench identifies tool misuse as a failure mode in 47.5% of Cy-Agent zero-day attempts [2]; PentestEval documents four specific failure patterns in exploit generation — missing parameters, loss of critical payload syntax, hallucinated dependencies, and incorrect tool flags [3]. These failures occur at the Execution/Validation Layer and will directly affect any performance number this thesis reports. *Mitigation:* the Execution/Validation Layer will include a retry-and-diagnose loop that captures tool-call errors, classifies them by failure type, and feeds structured error context back to the generating Specialist before counting a failure — consistent with PentestEval's Exploit Revision stage design.

4. **Context loss and recency bias across long missions.** Deng et al. [9] document two compounding failure modes in long-session agents: context loss (the agent loses awareness of previously attempted paths as the conversation grows) and depth-first recency bias (the agent anchors to the most recently discussed surface and fails to return to earlier discoveries). Any multi-phase VAPT mission of realistic length will trigger both. *Mitigation:* the Dual-Layer World Model separates confirmed facts (Evidence Layer, persistent) from inferred hypotheses (Analysis Layer/VDG, updated incrementally) so that context compaction in the LLM's active window does not corrupt the agent's factual record of what has been attempted.

5. **Benchmark scale varies significantly across surfaces.** PrediQL covers only 6 GraphQL APIs, against CVE-Bench's 40 CVEs and MHBench's 40 environments. PrediQL's APIs are real production deployments (not synthetic),

which gives them high ecological validity, but the small N limits statistical confidence. *Mitigation:* conclusions drawn from PrediQL results will be labelled exploratory; confidence intervals will be reported where sample size permits; and PrediQL findings will be presented as hypothesis-generating rather than hypothesis-confirming.

6. **Reused strategies could introduce negative transfer.** A Cross-Mission Memory entry for a successful exploit against one version of a technology could be actively harmful if replayed against an incompatible version — for example, a payload crafted for WordPress 5.9 breaking against WordPress 6.4. The reviewed papers do not address this risk. *Mitigation:* Cross-Mission Memory entries will store the target technology fingerprint (application name, version range, framework) alongside the strategy; replay will be gated by a version-compatibility check before the strategy is offered to the Team Manager.

7. **Sensitivity to configuration choices.** The VDG's UCB exploration constant, the edge-confidence pruning threshold, and the Evidence Layer's staleness window are all tunable parameters. Reporting results from a single, un-swept configuration risks overfitting to a fortunate setting. *Mitigation:* a sensitivity analysis will be conducted on at minimum the UCB exploration constant and the VDG edge-confidence threshold before the final evaluation configuration is fixed; the swept range and chosen value will be reported explicitly.

8. **Honesty about what generalises across surfaces.** Testing across web, GraphQL, and multi-host surfaces with the same system does not mean the entire system is unmodified across surfaces. Specialist Agent pools, tool adapters (ZAP/sqlmap for web; schema-aware fuzzer for GraphQL; BloodHound/credential tools for Active Directory), and oracle interfaces will necessarily differ. *Mitigation:* per-surface results will be reported separately, and a component table will explicitly identify which modules are shared across all surfaces and which are surface-specific, so that claims about generalisation are bounded by what is actually shared.

1.7 Organization

This report reflects the first stage of work on this thesis and its organization serves as a plan for the full document to be submitted over the remaining thesis period, not a description of chapters that already exist in complete form. The remainder of the report is structured as follows.

Chapter 2 reviews eleven papers directly relevant to autonomous LLM-based penetration testing, organised by contribution category: early single-agent systems, multi-agent and team-based orchestration, planning-augmented approaches, non-web attack surfaces, and the two benchmark studies — CVE-Bench [2] and PentestEval [3] — that provide the quantitative failure-mode evidence motivating this thesis. A comparative analysis identifies where the reviewed literature addresses exploration breadth and dependency reasoning separately but not jointly, and the chapter closes with the research gap that Sections 1.2 and 1.3 above translate into concrete research questions.

Chapter 3 presents the proposed RedGrid architecture as it develops. The core design consists of four components under active specification: a four-layer orchestration hierarchy (Mission Planner, Team Manager, Specialist Agents, and Execution/Validation Layer); a Dual-Layer World Model separating confirmed evidence from inferred hypotheses; a Vulnerability Dependency Graph (VDG) with UCB-guided node selection governing exploration; and a Cross-Mission Memory module for selective strategy reuse across engagements. This chapter is subject to revision as implementation proceeds and early experiments return feedback.

Chapter 4 lays out the work breakdown, timeline, and milestones for the remaining months of the thesis, and will be updated as work progresses to reflect completed and deferred items.

Chapter 5 specifies the evaluation plan and, once an implementation exists, will report results against the four governing benchmarks identified in Section 1.4: CVE-Bench, PentestEval, PrediQL, and Incalmo MHBench. Per-surface reporting and baseline comparisons are described in advance of any results being available.

Chapter 6 summarises the completed work, states which hypotheses the experimental results support or refute, and identifies directions for future work. That chapter takes substantive form only once the evaluation is complete.

Chapter 2

Literature Review and Related Work

2.1 Domain Overview

This chapter reviews the literature on LLM-based autonomous penetration testing and identifies the patterns and gaps that motivate the research direction described in Chapter 1. The review covers eleven papers selected as the most directly relevant to the research questions established in Chapter 1. Table 2.1 provides an overview; the sections that follow discuss each paper in more depth.

Autonomous penetration testing — the use of LLM agents to independently discover and exploit weaknesses in a target system without step-by-step human direction — has emerged as an active research area since approximately 2023, when Deng et al. [9] published the first systematic study of LLM-assisted penetration testing. The domain sits at the intersection of two older fields: Vulnerability Assessment and Penetration Testing (VAPT), which traditionally relies on human operators following structured methodologies (reconnaissance, weakness identification, exploitation, and reporting), and LLM agent research, which studies how language models can plan, use tools, and act toward a goal over sequential steps. RedGrid, the system described in this report, sits in this intersection: its focus is how an LLM-driven

multi-agent system can carry out the VAPT workflow autonomously against realistic, benchmarked targets.

A useful way to frame the domain is around three capabilities that an autonomous VAPT agent must exhibit. The first is *exploration*: the agent must search a large space of candidate weaknesses across an unfamiliar attack surface without prior knowledge of which one is exploitable. CVE-Bench [2] measures this directly and finds exploration failure rates of 37.5%–80.0% across every tested agent. The second is *prerequisite reasoning*: many real exploits are not single actions but chains, where one weakness must precede a second before it becomes reachable. PentestEval [3] measures this and finds that attack decision-making — reasoning about which weakness to pursue next given what the agent has discovered — is the weakest stage across nine evaluated LLMs, with an average Spearman correlation of 0.25 against expert judgement. The third is *cross-surface generalisability*: whether the same architecture can operate effectively across qualitatively different attack surfaces (web applications, GraphQL APIs, multi-host networks) without surface-specific redesign. As the literature reviewed in this chapter shows, most systems address at most one of these three capabilities; none of the reviewed papers evaluates the same architecture across more than one attack-surface family.

The sections that follow survey the systems and benchmarks from this selection (Section 2.2), draw out the recurring architectural patterns across them (Section 2.3), and identify where the reviewed literature leaves the compound gap that this thesis investigates (Section 2.4).

2.2 Existing Systems

This section discusses the eleven papers selected in Table 2.1, grouped by the category of contribution they represent.

TABLE 2.1: Eleven papers selected for review, with their main focus and key relevance to this research.

No.	Paper	Main Focus	Key Result / Relevance
1	Fang et al. (2024) — LLM Agents can Autonomously Hack Websites [8]	Single-agent web exploitation	First autonomous web attack demonstration; 73.3% sandbox vs. 2% real-world XSS gap
2	Fang et al. (2024) — LLM Agents can Autonomously Exploit One-day Vulnerabilities [5]	Single-agent CVE exploitation	GPT-4: 87% pass@5 with CVE desc., 7% without; establishes exploiting $\neq$ discovering
3	Zhu et al. (2024) — Teams of LLM Agents can Exploit Zero-Day Vulnerabilities [1]	Hierarchical multi-agent VAPT	First zero-day autonomous exploitation; 42% pass@5 / 18% pass@1 on 14-CVE suite; introduces planner + specialist design
4	Deng et al. (2024) — PentestGPT [9]	Single-agent with reasoning/parsing split	Identifies context-loss (Finding 3) and depth-first recency bias (Finding 4); motivates Dual-Layer World Model
5	Kong et al. (2025) — VulnBot [4]	Multi-agent, role-specialised	Representative team-dispatch architecture; flat task dispatch without prerequisite modeling
6	Wang et al. (2025) — CHECKMATE [10]	LLM agents + classical planning	Explicit PDDL-style prerequisite modeling; requires pre-enumerated weakness set; curated scenarios only
7	Singer et al. (2025) — Incalmo [7]	Multi-host LLM red teaming	Extends VAPT to multi-host / Active Directory; MHBench (40 environments); declarative task API
8	Liu et al. (2025) — PrediQL [11]	GraphQL API testing with LLMs	Schema-derived dependency reasoning on 6-API benchmark; non-web attack surface
9	Zhu et al. (2025) — CVE-Bench [2]	Benchmark: web CVE exploitation	40 critical CVEs; best agent 13% pass@5; insufficient exploration dominant failure (37.5%–80%)
10	Yang et al. (2025) — PentestEval [3]	Stage-level benchmark	346 tasks; ADM weakest stage (avg. Spearman 0.25); GT-ADM ablation raises success 0.31$\rightarrow$0.67
11	Wang et al. (2025) — Survey on LLM-based Autonomous Agents [6]	General LLM agent survey	Architecture > model scale finding; conceptual vocabulary for all domain-specific systems

2.2.1 Early Single-Agent Approaches

The earliest work in this space used a single LLM agent operating in a Reason + Act (ReAct) loop against a target. Fang et al. [8] showed that a single GPT-4 agent with browser access could autonomously exploit simplified web vulnerabilities, establishing that the basic capability exists but noting a substantial gap between sandbox success (73.3% XSS exploitation rate) and real-world success (2% across 50 candidate production sites). A later study by the same group [5] collected a benchmark of 15 real-world one-day vulnerabilities and found that GPT-4, when given the CVE description, achieved a pass@5 success rate of 87% — while eight other tested LLMs (including GPT-3.5) and two open-source scanners (ZAP and Metasploit) achieved 0%. Crucially, without the CVE description, GPT-4's pass@5 rate collapsed to 7%, establishing a distinction that motivates most of the later work in the field: exploiting a known vulnerability — one whose location and nature the agent already has — is a qualitatively different problem from discovering an unknown vulnerability independently.

Deng et al. [9] extended single-agent design with an explicit reasoning/generation/-parsing split and a Penetration Testing Tree (PTT) state structure aimed at reducing two documented failure modes: context loss over long testing sessions, in which the agent loses awareness of previously attempted paths as the conversation grows (Finding 3), and depth-first recency bias, in which the agent anchors to the most recently discussed surface and fails to return to earlier discoveries (Finding 4). PentestGPT operates as a human-in-the-loop assistant rather than a fully autonomous agent — human operators execute its suggested commands and return results — which limits the direct comparability of its results to fully autonomous systems. The fundamental tension it identifies, however, is relevant to any long-horizon agent: maintaining awareness of the full attack surface across a long mission requires context that grows without bound, while LLM reasoning quality degrades as context length increases. The Dual-Layer World Model in RedGrid is a direct architectural response to this tension.

2.2.2 Multi-Agent and Team-Based Orchestration

A second line of work replaces the single agent with a team of cooperating agents, motivated by the observation that a single flat loop struggles to scale to wide or unfamiliar attack surfaces. Zhu et al. [1] introduced HPTSA, a three-tier hierarchy consisting of a high-level planner, a subagent-creator that instantiates task-specific agents on demand, and multiple task-specific subagents each responsible for a single vulnerability type. HPTSA was the first system reported to exploit real zero-day vulnerabilities autonomously, achieving a pass@5 rate of 42% and a pass@1 rate of 18% on a 14-CVE zero-day benchmark — a 4.3× improvement over a single-agent baseline on pass@1. The result demonstrates that, with the right architecture, LLM agents can discover vulnerabilities rather than merely exploit known ones.

Kong et al. [4] propose VulnBot, a multi-agent framework with explicit role specialisation organised around a **Penetration Testing Graph (PTG)** — a structured task representation that sequences the penetration testing workflow into phases (reconnaissance, scanning, exploitation) and assigns phase-specific agents to each node. This structured dispatch is a step beyond flat sequencing. However, VulnBot does not model prerequisite relationships *between candidate vulnerabilities within a phase*: when multiple candidate weaknesses are identified at the exploitation phase, they are dispatched without a formal model of which must precede others. This is the limitation that distinguishes VulnBot from a dependency-aware system.

A point that emerges consistently from these multi-agent systems, and that the broader survey of the field supports [6], is that **architecture, not model scale, is the dominant variable in autonomous agent performance**. Singer et al. [7] provide a direct empirical demonstration: their multi-host framework running Claude Haiku 3.5 surpasses a strong single-agent baseline running Claude Sonnet 4 on MHBench, a more capable model. A well-structured pipeline with a weaker model outperforms an unstructured loop with a stronger one. This finding shapes the structural priorities of RedGrid.

2.2.3 Planning-Augmented Approaches

A smaller set of papers pairs LLM agents with a more classical planning component, aiming to give the agent an explicit model of prerequisites between actions rather than relying purely on the LLM's implicit reasoning. Wang et al. [10] combine LLM agents with PDDL-style planning operators for penetration testing, and report stronger performance on structured scenarios where the planning layer applies. The system's specific limitation is its entry condition: relevant weaknesses must be enumerated by the operator before the planning phase begins. CHECKMATE's planning is applied to a pre-supplied weakness set, not to weaknesses the agent discovers dynamically from an unfamiliar surface. This restricts its applicability to open-ended VAPT against unknown targets, a limitation discussed further in Section 2.4.

2.2.4 Non-Web Attack Surfaces and Benchmarks

The final group covers two non-web attack surfaces and the two benchmark studies that provide the quantitative evidence base for this thesis.

Singer et al. [7] extend autonomous penetration testing to multi-host and Active Directory-style network environments, where the agent must perform lateral movement, credential reuse, and privilege escalation across a chain of networked hosts with varying access states. This is a qualitatively different challenge from single-target web testing: the agent must maintain a graph of compromised and uncompromised hosts and reason about reachability across the network as its access state evolves. Incalmo addresses this through a declarative task vocabulary and a finite-state machine governing agent transitions; MHBench — the evaluation suite introduced alongside Incalmo — covers 40 networked environments across three difficulty tiers. The architecture-primacy result noted above (Haiku 3.5 outperforming Sonnet 4) was observed on MHBench.

Liu et al. [11] target GraphQL APIs specifically. Their key contribution is schema-derived dependency reasoning: a GraphQL schema encodes relationships between

types and fields, and PrediQL uses this structure to guide which queries to fuzz and in what order, rather than treating the API as an opaque surface. Their 6-API benchmark covers real production GraphQL deployments and shows that schema-guided agents outperform schema-agnostic baselines on dependency-chain vulnerabilities — the same class of reasoning gap that PentestEval identifies at the stage level.

On the evaluation side, Zhu et al. [2] introduce CVE-Bench, a benchmark of 40 critical-severity (CVSS ≥ 9.0) real-world web-application CVEs with oracle-backed pass/fail signals. Its failure-mode analysis provides the clearest available quantitative evidence that *insufficient exploration* — not reasoning quality per se — forms the primary bottleneck: exploration-failure rates range from 37.5% to 80.0% across every agent and evaluation setting, and the best-performing agent achieves only 13% pass@5. Yang et al. [3] introduce PentestEval, which decomposes the penetration testing pipeline into six stages and evaluates nine LLMs on each. The two weakest stages are Weakness Gathering (Jaccard 0.29) and Attack Decision-Making (Spearman correlation 0.25), the latter being the planning stage requiring prerequisite reasoning between candidate weaknesses. The ground-truth-injection ablation confirms that replacing ADM output with expert annotations yields the largest single-stage end-to-end gain (+0.14, raising the overall success rate from 0.31 to 0.67). Taken together, CVE-Bench and PentestEval identify the same compound problem from two angles: existing systems fail both at finding the right attack surface (exploration) and at reasoning about which discovered weakness to pursue in which order (prerequisite reasoning).

2.3 Comparative Analysis

Table 2.2 summarises the eleven reviewed systems and benchmarks along four dimensions that recurred throughout the literature review: agent architecture, dependency modeling approach, cross-session memory, and evaluation benchmark. The

patterns visible in this table, and a fifth pattern not captured by it, are discussed in the analysis that follows.

TABLE 2.2: Comparison of systems and benchmarks reviewed, across four recurring architectural dimensions.

System / Work	Architecture	Dependency Modeling	Cross-Session Memory	Benchmark
Fang et al. [8]	Single agent (ReAct)	None	None	Simplified websites
Fang et al. [5]	Single agent (ReAct)	None (CVE desc. provided externally)	None	15 one-day CVEs
HPTSA [1]	3-tier: planner + subagent-creator + task agents	None (flat vuln. dispatch)	None	14 zero-day CVEs
PentestGPT [9]	Single agent, 3-module split (human-assisted)	Implicit (LLM reasoning only)	None	HTB / VulnHub machines
VulnBot [4]	Multi-agent, PTG phase-sequenced	PTG phases; no vuln-level pre-requisite model	None	HTB-style scenarios
CHECKMATE [10]	Agent + classical PDDL planner	Explicit, pre-enumerated by operator	None	Curated scenarios
Incalmo [7]	Multi-host FSM orchestration	Partial (host / credential graph)	None	MHBench (40 envs.)
PrediQL [11]	LLM-guided schema fuzzer	Schema-derived (GraphQL field graph)	None	6 GraphQL APIs
CVE-Bench [2]	Benchmark (evaluates agents)	N/A	N/A	40 critical web CVEs
PentestEval [3]	Benchmark (modular pipeline + GT ablation)	None (SMP); GT-injected in ablation only	None	12 real-world scenarios
Wang et al. [6]	Survey	N/A	N/A	General (survey)

Four patterns emerge from this comparison, and a fifth is notable for being absent.

Pattern 1 — Exploration-capable systems lack dependency modeling. Systems that scale to wide or unfamiliar attack surfaces — HPTSA and VulnBot

— dispatch vulnerability candidates without an explicit model of which weaknesses must precede others. HPTSA dispatches subagents in parallel with no prerequisite ordering; VulnBot sequences tasks by pentesting phase via its PTG but does not model dependency relationships between individual candidate vulnerabilities within a phase.

Pattern 2 — Dependency-modeling systems require pre-supplied weakness sets. Two systems model dependencies explicitly: CHECKMATE, using PDDL-style planning operators, and PrediQL, using the GraphQL schema as a dependency graph. Both operate on externally provided structure. CHECKMATE requires the operator to enumerate relevant weaknesses before the planning phase begins; it is not designed for open-ended discovery on an unknown surface. PrediQL derives its dependency graph from the GraphQL schema, which is provided by the API itself. Neither system constructs a dependency model dynamically from its own discovery output.

Pattern 3 — Cross-session memory is universally absent. Every autonomous system in the table retains no information across separate missions. Each engagement begins from scratch regardless of what the agent discovered in prior engagements against the same or similar targets. This gap is not addressed in any reviewed paper.

Pattern 4 — Both benchmark studies converge on the same compound failure. CVE-Bench's failure-mode analysis identifies *insufficient exploration* as the dominant failure (37.5%–80.0% of unsuccessful attempts, depending on agent and setting). PentestEval's stage-level breakdown identifies *attack decision-making* — reasoning about which discovered weakness to pursue next, in what order — as the weakest planning stage (average Spearman correlation 0.25 across nine LLMs). The two studies approach the same system from opposite ends and identify the two sides of the same underlying gap.

Pattern 5 (notable absence) — No cross-surface evaluation. Every autonomous system in the table undergoes evaluation against a single attack-surface family: web CVEs, GraphQL APIs, or multi-host networks, but never more than

one. This means that architectural decisions cannot be distinguished from surface-specific tuning effects in any existing study.

The following section discusses what these patterns imply for the research direction of this thesis.

2.4 Research Gap

The comparative analysis in Section 2.3 establishes that autonomous VAPT systems have so far addressed at most one of three capabilities at a time. Table 2.3 consolidates this finding across the reviewed literature.

TABLE 2.3: Three-dimensional gap in the reviewed autonomous VAPT literature: what each capability area shows and where it falls short.

Capability area	**What the reviewed literature shows**	**Documented limitation**
Exploration breadth	HPTSA and VulnBot scale to wide, unfamiliar attack surfaces through hierarchical team dispatch and phase-sequenced PTG planning	Neither models prerequisite relationships between discovered vulnerabilities; no formal notion of which weakness must precede another
Prerequisite / dependency reasoning	CHECKMATE introduces PDDL-style explicit prerequisite modeling; PentestEval's GT-ADM ablation shows that expert-quality ADM raises end-to-end success from 0.31 to 0.67 (+0.14)	Both require weaknesses to be pre-enumerated by the operator before planning begins; neither constructs a dependency model from dynamic discovery output
Cross-session memory	No system in the reviewed set retains or reuses strategies across engagements against the same or similar targets	An open area this thesis treats as secondary to the above two gaps; empirical value to be assessed once the primary gaps are addressed
Cross-surface evaluability	Every autonomous system in the reviewed set evaluates on a single attack-surface family	Architectural decisions cannot be distinguished from surface-specific tuning effects in any existing study

On one side, the systems built for broad exploration of an unfamiliar attack surface — HPTSA and VulnBot — do not model prerequisite relationships between

candidate weaknesses. They search widely, but without a formal notion of which weaknesses must precede others; when multiple candidates are identified, they are dispatched without an ordering that reflects dependency structure. On the other side, the one system that does model vulnerability-level prerequisites explicitly — CHECKMATE, using PDDL-style planning operators — requires the operator to enumerate relevant weaknesses before the planning phase begins. CHECKMATE is designed for structured scenarios with a supplied weakness set, not for open-ended discovery on an unfamiliar surface where no such set exists in advance. PrediQL introduces schema-derived dependency reasoning, but this is derived from the GraphQL schema provided by the API itself, not from the agent's dynamic discovery output; and PrediQL is surface-specific to GraphQL by design.

The two benchmark studies in this selection quantify the cost of each missing capability. CVE-Bench's failure-mode analysis finds that insufficient exploration accounts for 37.5%–80.0% of unsuccessful attempts across every agent and evaluation setting; its best-performing agent achieves only 13% pass@5 [2]. PentestEval's stage-level breakdown finds that attack decision-making — the stage that requires reasoning about which discovered weakness to pursue in which order — achieves the lowest average performance of any planning stage (Spearman correlation 0.25 across nine evaluated LLMs) and delivers the largest single-stage improvement when replaced with expert ground truth (+0.14, raising end-to-end success from 0.31 to 0.67) [3]. Taken together, the two studies measure the same compound gap from opposite ends: CVE-Bench shows the cost of failing to find the right attack path; PentestEval shows the cost of failing to reason about the right next step once candidate weaknesses are known.

The gap this thesis investigates is therefore precise: no reviewed system combines (i) open-ended exploration of an unfamiliar attack surface with (ii) a dynamically constructed, prerequisite-aware model of dependencies between discovered weaknesses, and (iii) evaluation across more than one independently benchmarked attack-surface family. Chapter 3 introduces the proposed RedGrid architecture as a first attempt to address this compound gap.

2.5 Summary

This chapter has reviewed eleven papers directly relevant to autonomous, LLM-driven penetration testing, covering early single-agent systems (Fang et al., PentestGPT), multi-agent orchestration frameworks (HPTSA, VulnBot), planning-augmented approaches (CHECKMATE), non-web attack surfaces (Incalmo, PrediQL), and the two benchmark studies that provide the quantitative evidence base for the field's current limitations (CVE-Bench, PentestEval). The comparative analysis in Section 2.3 identified five structural patterns across this literature: exploration-capable systems lack vulnerability-level dependency modeling; dependency-modeling systems require pre-enumerated weakness sets and do not operate on dynamically discovered output; cross-session memory is absent from every reviewed system; the two benchmark studies converge on the same compound failure from opposite ends; and no autonomous system in the reviewed set has been evaluated across more than one attack-surface family.

Section 2.4 translates these patterns into a precise three-dimensional gap: no reviewed system combines open-ended exploration of an unfamiliar attack surface with a dynamically constructed prerequisite-aware dependency model and evaluation across independently benchmarked attack-surface families. This gap is the problem this thesis investigates. Chapter 3 introduces the proposed RedGrid architecture — the four-layer orchestration hierarchy, Dual-Layer World Model, and VDG — as a first design response to this gap. The architecture is at an early stage of specification and will receive further justification as implementation proceeds.

Chapter 3

Proposed Methodologies

3.1 System Overview

3.2 System Requirements

3.3 Proposed Architecture

3.4 System Workflow

3.5 Tools and Technologies

Chapter 4

Execution Plan

4.1 Work Breakdown Structure

4.2 Project Timeline

4.3 Milestones

4.4 Resource Requirements

4.5 Risk Assessment

Chapter 5

Experimental Results

5.1 Evaluation Plan

Chapter 6

Conclusions

6.1 Summary

6.2 Expected Deliverables

6.3 Future Works

Bibliography

[1] Yuxuan Zhu, Antony Kellermann, Akul Gupta, Philip Li, Richard Fang, Rohan Bindu, and Daniel Kang. Teams of LLM agents can exploit zero-day vulnerabilities. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2026. arXiv:2406.01637.

[2] Yuxuan Zhu, Antony Kellermann, Dylan Bowman, Philip Li, Akul Gupta, Adarsh Danda, Richard Fang, Conner Jensen, Eric Ihli, Jason Benn, Jet Geronimo, Avi Dhir, Sudhit Rao, Kaicheng Yu, Twm Stone, and Daniel Kang. CVE-Bench: A benchmark for AI agents' ability to exploit real-world web application vulnerabilities. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025.

[3] Ruozhao Yang, Mingfei Cheng, Gelei Deng, Tianwei Zhang, Junjie Wang, and Xiaofei Xie. PentestEval: Benchmarking LLM-based penetration testing with modular and stage-level design. Preprint, 2025.

[4] He Kong, Die Hu, Jingguo Ge, Liangxiong Li, Tong Li, and Bingzhen Wu. VulnBot: Autonomous penetration testing for a multi-agent collaborative framework. arXiv preprint arXiv:2501.13411, 2025.

[5] Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. LLM agents can autonomously exploit one-day vulnerabilities. arXiv preprint arXiv:2404.08144, 2024. v2, cs.CR.

[6] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhi-Yuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei

Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2025. arXiv:2308.11432.

[7] Brian Singer, Keane Lucas, Lakshmi Adiga, Meghna Jain, Lujo Bauer, and Vyas Sekar. Incalmo: An autonomous LLM-assisted system for red teaming multi-host networks. arXiv preprint arXiv:2501.16466, 2025.

[8] Richard Fang, Rohan Bindu, Akul Gupta, Qiusi Zhan, and Daniel Kang. LLM agents can autonomously hack websites. arXiv preprint arXiv:2402.06664, 2024.

[9] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*, pages 847–864, Philadelphia, PA, 2024. USENIX Association.

[10] Lingzhi Wang, Xinyi Shi, Ziyu Li, Yi Jiang, Shiyu Tan, Yuhao Jiang, Junjie Cheng, Wenyuan Chen, Xiangmin Shen, Zhenyuan Li, and Yan Chen. Automated penetration testing with LLM agents and classical planning. arXiv preprint arXiv:2512.11143, 2025.

[11] Shaolun Liu, Sina Marefat, Omar Tsai, Yu Chen, Zecheng Deng, Jia Wang, and Mohammad A. Tayebi. PrediQL: Automated testing of GraphQL APIs with LLMs. arXiv preprint arXiv:2510.10407, 2025.